```
from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
# Import Library
import pandas as pd
import numpy as np
# Visualisasi
import matplotlib.pyplot as plt
import seaborn as sns
# Preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import label_binarize
# Model
from sklearn.ensemble import RandomForestClassifier
# Evaluasi
from sklearn.metrics import {
    accuracy_score,
    classification_report,
    confusion_matrix,
    roc_curve,
    auc,
    roc_auc_score
}
```

EDA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('Students Social Media Addiction.csv')
print(df.shape)
df.head(705)
```

```
(705, 13)
```

	Student_ID	Age	Gender	Academic_Level	Country	Avg_Daily_Usage_Hours	Most_Used_Platform	Affects_Academic_Performance
0	1	19	Female	Undergraduate	Bangladesh	5.2	Instagram	Yes
1	2	22	Male	Graduate	India	2.1	Twitter	No
2	3	20	Female	Undergraduate	USA	6.0	TikTok	Yes
3	4	18	Male	High School	UK	3.0	YouTube	No
4	5	21	Male	Graduate	Canada	4.5	Facebook	Yes
...	...	...	...	...	...	...	...	...
700	701	20	Female	Undergraduate	Italy	4.7	TikTok	No
701	702	23	Male	Graduate	Russia	6.8	Instagram	Yes
702	703	21	Female	Undergraduate	China	5.6	WeChat	Yes
703	704	24	Male	Graduate	Japan	4.3	Twitter	No
704	705	19	Female	Undergraduate	Poland	6.2	Facebook	Yes

705 rows × 13 columns

```
print("informasi data set :")
df.info()

print("\nJumlah Data Duplikat :", df.duplicated().sum())
print("Jumlah Missing Values :\n", df.isnull().sum())
print("\nStatistik Deskriptif :")
df.describe()
```

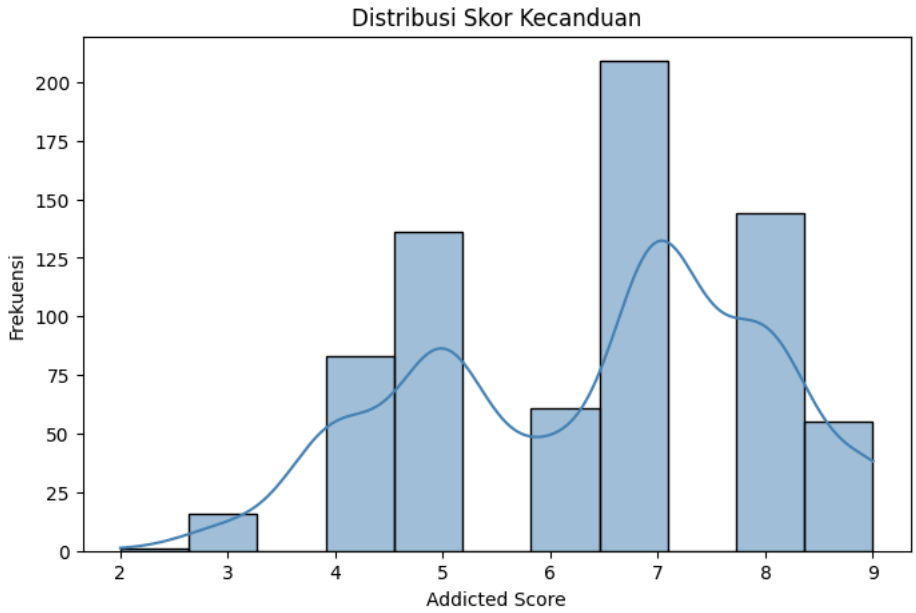
```
informasi data set :
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 705 entries, 0 to 704
Data columns (total 13 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   Student_ID                    705 non-null    int64
 1   Age                           705 non-null    int64
 2   Gender                        705 non-null    object
 3   Academic_Level                705 non-null    object
 4   Country                       705 non-null    object
 5   Avg_Daily_Usage_Hours         705 non-null    float64
 6   Most_Used_Platform            705 non-null    object
 7   Affects_Academic_Performance  705 non-null    object
 8   Sleep_Hours_Per_Night         705 non-null    float64
 9   Mental_Health_Score           705 non-null    int64
 10  Relationship_Status           705 non-null    object
 11  Conflicts_Over_Social_Media   705 non-null    int64
 12  Addicted_Score                705 non-null    int64
dtypes: float64(2), int64(5), object(6)
memory usage: 71.7+ KB

Jumlah Data Duplikat : 0
Jumlah Missing Values :
 Student_ID                      0
Age                              0
Gender                           0
Academic_Level                   0
Country                          0
Avg_Daily_Usage_Hours            0
Most_Used_Platform               0
Affects_Academic_Performance     0
Sleep_Hours_Per_Night            0
Mental_Health_Score              0
Relationship_Status              0
Conflicts_Over_Social_Media      0
Addicted_Score                   0
dtype: int64

Statistik Deskriptif :
```

	Student_ID	Age	Avg_Daily_Usage_Hours	Sleep_Hours_Per_Night	Mental_Health_Score	Conflicts_Over_Social_Media
count	705.000000	705.000000	705.000000	705.000000	705.000000	705.000000
mean	353.000000	20.659574	4.918723	6.868936	6.226950	2.849645
std	203.660256	1.399217	1.257395	1.126848	1.105055	0.957968
min	1.000000	18.000000	1.500000	3.800000	4.000000	0.000000
25%	177.000000	19.000000	4.100000	6.000000	5.000000	2.000000
50%	353.000000	21.000000	4.800000	6.900000	6.000000	3.000000
75%	529.000000	22.000000	5.800000	7.700000	7.000000	4.000000
max	705.000000	24.000000	8.500000	9.600000	9.000000	5.000000

```
#visualisasi dengan histogram distribusi target (addicted_score)
plt.figure(figsize=(8,5))
sns.histplot(df['Addicted_Score'], kde=True, color='steelblue')
plt.title('Distribusi Skor Kecanduan')
plt.xlabel('Addicted Score')
plt.ylabel('Frekuensi')
plt.show()
```



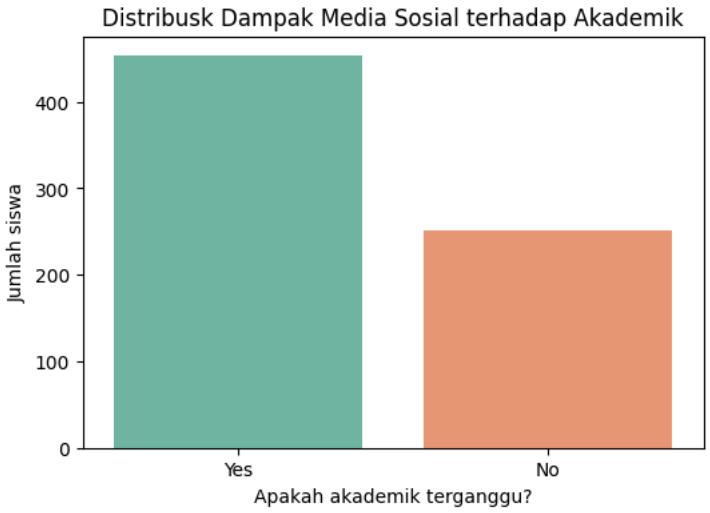

```
#dampak media sosial terhadap akademik
plt.figure(figsize=(6,4))
sns.countplot(x='Affects_Academic_Performance', data=df, palette='Set2')
plt.title('Distribusk Dampak Media Sosial terhadap Akademik')
plt.xlabel('Apakah akademik terganggu?')
plt.ylabel('Jumlah siswa')
plt.show()

#Kesimpulan: dari seluruh siswa yang didata, yang merasa nilai akademiknya terganggu (Yes)
# jumlahnya lebih banyak dibandingkan yang tidak terganggu (No)
```

```
/tmp/ipykernel_2219/701729501.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

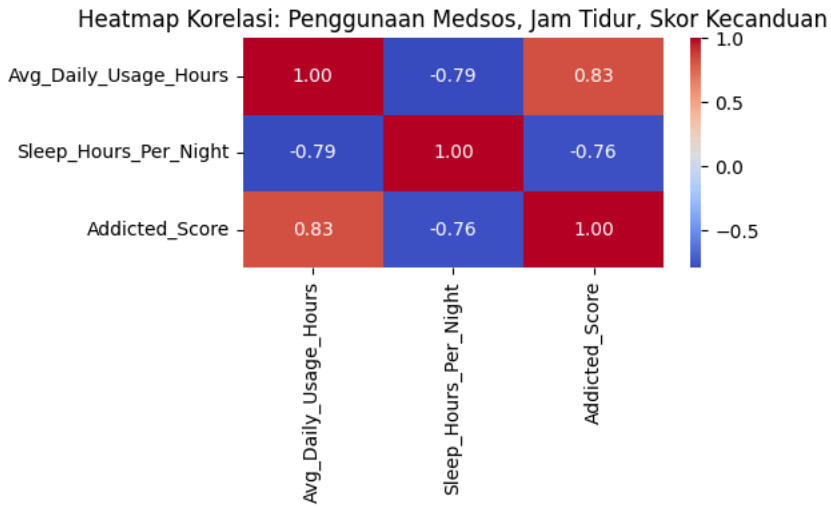
  sns.countplot(x='Affects_Academic_Performance', data=df, palette='Set2')
```




```
# Korelasi antara Jam Penggunaan, Tidur, dan Skor Kecanduan dengan heatmap korelasi
plt.figure(figsize=(6, 4))
cols = ['Avg_Daily_Usage_Hours', 'Sleep_Hours_Per_Night', 'Addicted_Score']
sns.heatmap(df[cols].corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Heatmap Korelasi: Penggunaan Medsos, Jam Tidur, Skor Kecanduan')
plt.tight_layout()
plt.show()

# Kesimpulan:
# - Avg_Daily_Usage_Hours vs Sleep_Hours_Per_Night → korelasi negatif:
#   semakin lama penggunaan medsos, jam tidur cenderung berkurang.
```

```
# - Avg_Daily_Usage_Hours vs Addicted_Score → korelasi positif:
#   semakin lama penggunaan harian, skor kecanduan semakin tinggi.
```




```
# Membandingkan Dampak Akademik dengan Skor Kecanduan
plt.figure(figsize=(8, 5))
sns.boxplot(x='Affects_Academic_Performance', y='Addicted_Score', data=df, palette='Set2')

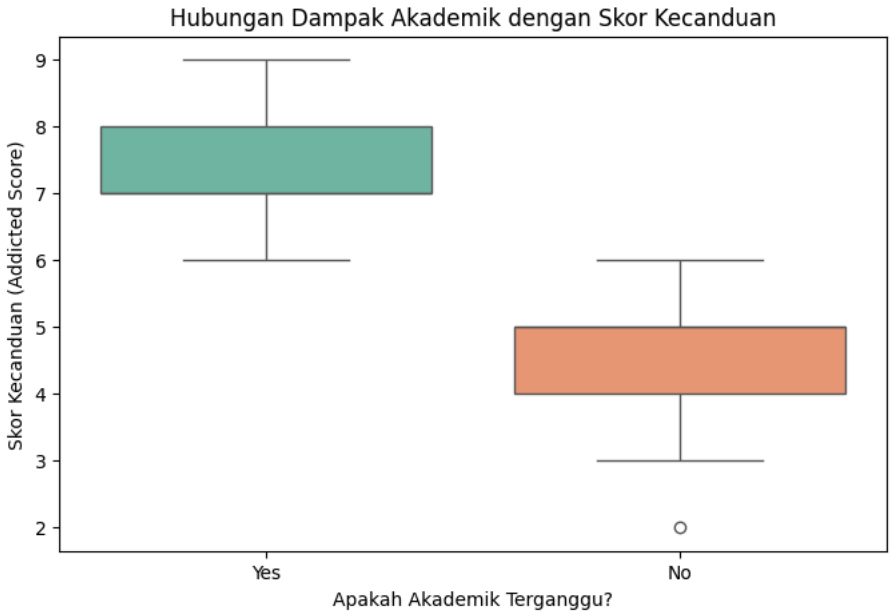
plt.title('Hubungan Dampak Akademik dengan Skor Kecanduan')
plt.xlabel('Apakah Akademik Terganggu?')
plt.ylabel('Skor Kecanduan (Addicted Score)')
plt.show()

# Kesimpulan:
# - Kelompok 'Yes' (akademik terganggu) memiliki median skor kecanduan lebih tinggi (~7-8)
# - Kelompok 'No' memiliki skor lebih rendah (~4-5)
# → Kecanduan media sosial berkorelasi positif dengan gangguan akademik
```

```
/tmp/ipykernel_2219/1380857362.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

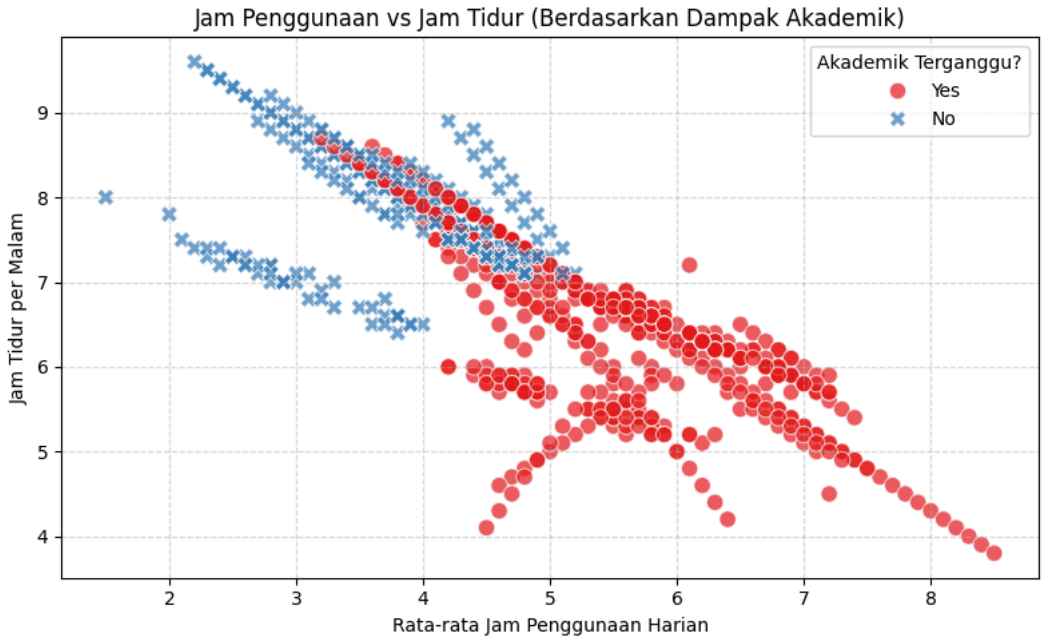
  sns.boxplot(x='Affects_Academic_Performance', y='Addicted_Score', data=df, palette='Set2')
```




```
# Hubungan antara jam penggunaan harian, jam tidur, dan dampak akademik
# Hipotesis: semakin lama penggunaan medsos → jam tidur berkurang → tidak fokus belajar → nilai menurun
plt.figure(figsize=(8, 5))
sns.scatterplot(
```

```
    x='Avg_Daily_Usage_Hours',
    y='Sleep_Hours_Per_Night',
    hue='Affects_Academic_Performance',
    style='Affects_Academic_Performance',
    palette='Set1',
    s=80,
    alpha=0.7,
    data=df
)

plt.title('Jam Penggunaan vs Jam Tidur (Berdasarkan Dampak Akademik)')
plt.xlabel('Rata-rata Jam Penggunaan Harian')
plt.ylabel('Jam Tidur per Malam')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(title='Akademik Terganggu?')
plt.tight_layout()
plt.show()
```



PRE PROCESSING

```
data = df.drop(
    ['Student_ID','Gender', 'Academic_Level','Country','Relationship_Status', 'Conflicts_Over_Social_Media'], axis=1
)

print("\nKolom setelah drop variabel yang tidak relevan :")
print(data.columns.tolist())
```

```
Kolom setelah drop variabel yang tidak relevan :
['Age', 'Avg_Daily_Usage_Hours', 'Most_Used_Platform', 'Affects_Academic_Performance', 'Sleep_Hours_Per_Night', 'Mental_Health_S
```

```
#mengkelompokan variabel addicted_score
def kelompokkan_skor(skor):
  if skor <=3:
    return 'Rendah'
  elif skor<=7:
      return 'Sedang'
  else:
        return 'Tinggi'

data['Addicted_Category'] = data['Addicted_Score'].apply(kelompokkan_skor)
data = data.drop('Addicted_Score', axis=1)

# VISUALISASI PASCA-TRANSFORMASI: Cek distribusi kelas target baru (Keseimbangan Kelas)
plt.figure(figsize=(7, 4))
order_kategori = ['Rendah', 'Sedang', 'Tinggi']
sns.countplot(x='Addicted_Category', data=data, palette='Set2', order=order_kategori)
plt.title('Distribusi Target Baru Setelah Proses Binning (Pre-processing)')
```

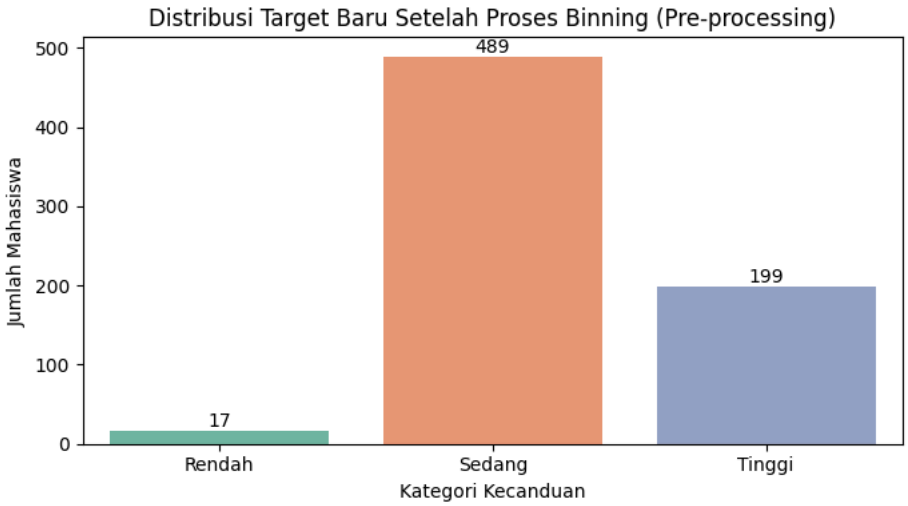
```
plt.xlabel('Kategori Kecanduan')
plt.ylabel('Jumlah Mahasiswa')

# Tampilkan nominal angka di atas setiap balok grafik
ax = plt.gca()
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}', (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='center', xytext=(0, 5), textcoords='offset points')
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_2219/210546153.py:16: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  sns.countplot(x='Addicted_Category', data=data, palette='Set2', order=order_kategori)
```




```
# Mengubah data kategori teks menjadi angka menggunakan One-Hot Encoding
data_encoded = pd.get_dummies(data, columns=['Most_Used_Platform'], drop_first=True)

# Encoding kolom Yes/No menjadi 1/0
data_encoded['Affects_Academic_Performance'] = (
    data_encoded['Affects_Academic_Performance'].map({'Yes': 1, 'No': 0})
)

# Verifikasi tidak ada nilai NaN setelah encoding (jika ada nilai selain Yes/No, map akan menghasilkan NaN)
print("Missing value setelah encoding:", data_encoded['Affects_Academic_Performance'].isnull().sum())
print("Tipe data setelah encoding:")
print(data_encoded.dtypes)
data_encoded.head()
```

```
Missing value setelah encoding: 0
Tipe data setelah encoding:
Age                              int64
Avg_Daily_Usage_Hours          float64
Affects_Academic_Performance     int64
Sleep_Hours_Per_Night          float64
Mental_Health_Score              int64
Addicted_Category               object
Most_Used_Platform_Instagram      bool
Most_Used_Platform_KakaoTalk      bool
Most_Used_Platform_LINE           bool
Most_Used_Platform_LinkedIn       bool
Most_Used_Platform_Snapchat       bool
Most_Used_Platform_TikTok         bool
Most_Used_Platform_Twitter        bool
Most_Used_Platform_VKontakte      bool
Most_Used_Platform_WeChat         bool
Most_Used_Platform_WhatsApp       bool
Most_Used_Platform_YouTube        bool
dtype: object
```

	Age	Avg_Daily_Usage_Hours	Affects_Academic_Performance	Sleep_Hours_Per_Night	Mental_Health_Score	Addicted_Category	Most_
0	19	5.2	1	6.5	6	Tinggi	
1	22	2.1	0	7.5	8	Rendah	
2	20	6.0	1	5.0	5	Tinggi	
3	18	3.0	0	7.0	7	Sedang	
4	21	4.5	1	6.0	6	Sedang	

```
# Memisahkan Fitur (X) dan Target Kategori (y)
X = data_encoded.drop('Addicted_Category', axis=1)
y = data_encoded['Addicted_Category']
# Splitting data menjadi data training dan data testing
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
    stratify = y #menjaga proporsi kelas tetap seimbang
)


print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)
```

```
X_train: (564, 16)
X_test : (141, 16)
y_train: (564,)
y_test : (141,)
```

MODELLING

```
from sklearn.ensemble import RandomForestClassifier

#menjalankan model dengan 200 pohon keputusan
model = RandomForestClassifier
 (n_estimators=200, random_state=42, n_jobs=-1)

#menjalankan proses training
model.fit(X_train,y_train)
```

```
RandomForestClassifier
RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=42)
```

```
# Feature Scaling — diterapkan pada X_train dan X_test (BUKAN X secara keseluruhan)

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

# fit HANYA pada data training, transform keduanya
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
print("Scaling selesai.")
```

```
Scaling selesai.
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# ===================================
# MODEL RANDOM FOREST CLASSIFIER
# ===================================
rf_clf = RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1)
rf_clf.fit(X_train, y_train)  # menggunakan X_train (tanpa scaling, karena RF tidak memerlukannya)
y_pred_rf = rf_clf.predict(X_test)

print("Akurasi Random Forest :", accuracy_score(y_test, y_pred_rf))
```

```
Akurasi Random Forest : 0.9787234042553191
```

EVALUASI

```
from sklearn.metrics import accuracy_score, classification_report,confusion_matrix

print("Hasil evaluasi model random forest klasifikasi 3 kelas")
print(f"Akurasi akhir model: {accuracy_score(y_test, y_pred_rf):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf, target_names=order_kategori))

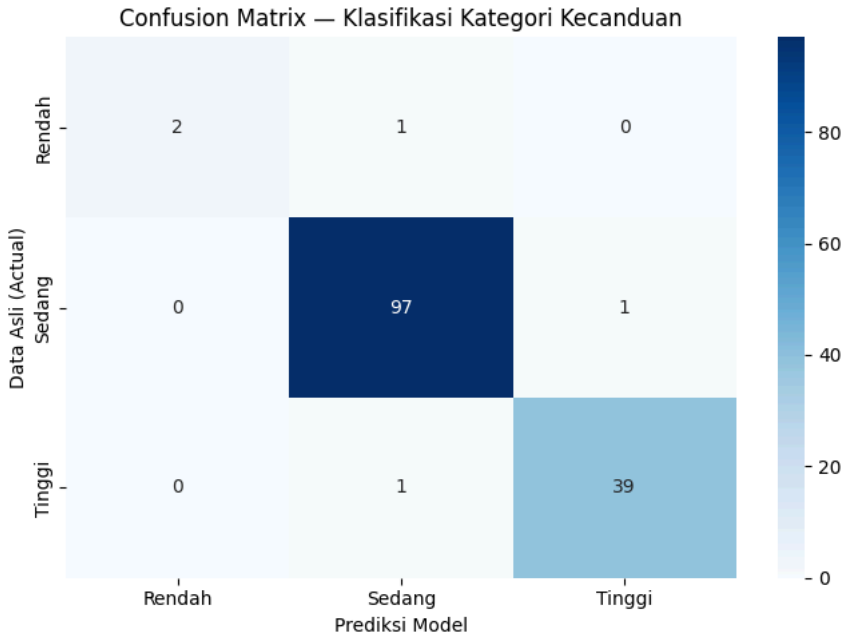
# VISUALISASI EVALUASI 1: Confusion Matrix 3x3 Lengkap dengan Label Sumbu
plt.figure(figsize=(7, 5))
cm = confusion_matrix(y_test, y_pred_rf, labels=order_kategori)
sns.heatmap(
    cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=order_kategori,
    yticklabels=order_kategori
)
plt.title('Confusion Matrix – Klasifikasi Kategori Kecanduan')
plt.xlabel('Prediksi Model')
plt.ylabel('Data Asli (Actual)')
plt.tight_layout()
plt.show()
```

```
Hasil evaluasi model random forest klasifikasi 3 kelas
Akurasi akhir model: 0.9787

Classification Report:
              precision    recall  f1-score   support

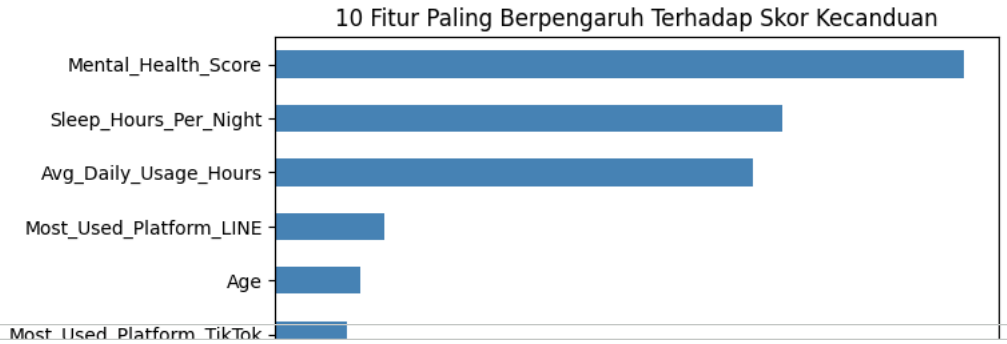
      Rendah       1.00      0.67      0.80         3
      Sedang       0.98      0.99      0.98        98
      Tinggi       0.97      0.97      0.97        40

    accuracy                           0.98       141
   macro avg       0.98      0.88      0.92       141
weighted avg       0.98      0.98      0.98       141
```




```
# Melihat fitur paling berpengaruh terhadap prediksi
plt.figure(figsize=(8, 5))
importances = pd.Series(rf_clf.feature_importances_, index=X.columns)
importances.nlargest(10).sort_values().plot(kind='barh', color='steelblue')
plt.title('10 Fitur Paling Berpengaruh Terhadap Skor Kecanduan')
plt.xlabel('Importance Score')
plt.tight_layout()
plt.show()

print("\nTop 10 Fitur:")
print(importances.nlargest(10).round(4))
```




```
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Probabilitas prediksi dari Random Forest
y_score = rf_clf.predict_proba(X_test)

# Binarisasi label multiclass
classes = ['Rendah', 'Sedang', 'Tinggi']
y_test_bin = label_binarize(y_test, classes=classes)

# Hitung ROC dan AUC
fpr = {}
tpr = {}
roc_auc = {}

for i in range(len(classes)):
    fpr[i], tpr[i], _ = roc_curve(
        y_test_bin[:, i],
        y_score[:, i]
    )
    roc_auc[i] = auc(fpr[i], tpr[i])

# Plot ROC Curve
plt.figure(figsize=(8,6))

for i in range(len(classes)):
    plt.plot(
        fpr[i],
        tpr[i],
        label=f'{classes[i]} (AUC = {roc_auc[i]:.3f})'
    )
```